

第 6 章 文件和数据格式

到现在为止，程序一关数据就没了。这一章把第二章的通讯录存进文件，下次打开还能读回来，顺便认识两种最常见的数据格式。

6.1 先把通讯录存下来

本节任务：把通讯录写进文件，程序重启后能原样读回来。

```
1 import json
2 from pathlib import Path
3
4 contacts = [
5     {"name": "小明", "phone": "13800001111"},
6     {"name": "小红", "phone": "13900002222"},
7 ]
8
9 path = Path("contacts.json")
10
11 # 存
12 path.write_text(json.dumps(contacts, ensure_ascii=False, indent=2),
13                 encoding="utf-8")
14
15 # 读
16 loaded = json.loads(path.read_text(encoding="utf-8"))
17 print("读回来", len(loaded), "条")
18 for p in loaded:
19     print(" ", p["name"], p["phone"])
```

运行结果：

```
已存到 D:\code\contacts.json
读回来 2 条
  小明 13800001111
  小红 13900002222
```

打开生成的 contacts.json 看一眼，内容是这样的，人也读得懂：

```
[
  {
```

```
"name": "小明",
"phone": "13800001111"
},
...
]
```

短短几行做完了存和读。下面把文件操作和数据格式分开讲。

6.2 读写文件

本节任务：会打开文件读内容、写内容，并知道为什么要指定编码。

最基本的读写用 `open`，配合 `with` 使用：

```
1 # 写，w 表示写入，原有内容会被覆盖
2 with open("note.txt", "w", encoding="utf-8") as f:
3     f.write("第一行\n")
4     f.write("第二行\n")
5
6 # 读，r 表示读取
7 with open("note.txt", "r", encoding="utf-8") as f:
8     content = f.read()
9     print(content)
10
11 # 一行一行地读
12 with open("note.txt", encoding="utf-8") as f:
13     for line in f:
14         print(line.strip()) # strip 去掉行尾的换行符
```

有两处要特别注意。一是 `with`，它保证文件用完自动关闭，哪怕中间出错也会关。不 ★核心用 `with` 就得自己记着调用 `close`，忘了可能导致内容没真正写进去。二是 `encoding` 要写 `utf-8`，尤其在 Windows 上。不写的话系统可能用别的编码，中文就会变成乱码或直接报错。

几种打开方式的区别：

表 6.1: `open` 的几种模式。

模式	含义
r	读，文件必须已存在
w	写，文件不存在就新建，已存在就清空重写
a	追加，在原有内容末尾接着写

w 会清空原文件，这是新手容易误伤数据的地方。想在末尾添加就用 a。

6.3 路径怎么写

本节任务：会拼路径、判断文件在不在，避开反斜杠的坑。

Windows 的路径用反斜杠分隔，但反斜杠在字符串里有特殊含义，直接写容易出问题。用 `pathlib` 就没这个烦恼，它会自动处理不同系统的差异：

```
1 from pathlib import Path
2
3 p = Path("data") / "contacts.json" # 用斜杠拼接，跨系统通用
4 print(p) # data\contacts.json 或 data/contacts.json
5 print(p.name) # contacts.json
6 print(p.suffix) # .json
7 print(p.exists()) # 文件在不在
8 print(p.resolve()) # 完整的绝对路径
9
10 Path("data").mkdir(exist_ok=True) # 建目录，已存在也不报错
```

`pathlib` 还能直接读写，省掉 `open`：

```
1 Path("note.txt").write_text("你好", encoding="utf-8")
2 print(Path("note.txt").read_text(encoding="utf-8"))
```

小文件用这两个方法最省事，开头存通讯录用的就是它们。

6.4 JSON，存有结构的数据

本节任务：把列表字典这类数据原样存进文件再读回来。

文本文件存的是纯文字，可列表和字典是有结构的。直接把字典写进文件，读回来就成了一串字符，还得自己解析。**JSON** 就是为解决问题而生的格式，它和 Python 的列表字典长得几乎一样。

```
1 import json
2
3 data = {"name": "小明", "scores": [86, 92], "passed": True}
4
5 s = json.dumps(data, ensure_ascii=False) # 转成字符串
6 print(s) # {"name": "小明", "scores": [86, 92], "passed": true}
7
```

```
8 back = json.loads(s) # 再转回来
9 print(back["scores"][0]) # 86
```

记住四个名字就够用了，**dumps** 转成字符串，**loads** 从字符串转回来，带 **s** 的这 ★核心一对处理字符串。还有一对不带 **s** 的直接和文件打交道：

```
1 with open("data.json", "w", encoding="utf-8") as f:
2     json.dump(data, f, ensure_ascii=False, indent=2)
3
4 with open("data.json", encoding="utf-8") as f:
5     back = json.load(f)
```

两个常用参数解释一下。`ensure_ascii` 设成 `False`，中文才会原样存，否则会变成一串看不懂的转义码。`indent` 设成 2 会自动换行缩进，文件打开后人也能读得懂，不写就挤成一行。

JSON 只认识几种类型，字符串、数字、布尔、列表、字典和空值。Python 里的元组会被存成列表，日期这类对象存不了，需要先转成字符串。

6.5 CSV，存表格数据

本节任务：读写表格文件，这是数据处理最常见的格式。

一批结构相同的记录，比如很多行成绩，更适合用 **CSV**。它本质是纯文本，每行一条记录，字段之间用逗号隔开，Excel 能直接打开。

```
1 import csv
2
3 rows = [
4     {"name": "小明", "chinese": 86, "math": 92},
5     {"name": "小红", "chinese": 95, "math": 88},
6 ]
7
8 # 写，newline 设成空字符串是固定写法，不然 Windows 上会多出空行
9 with open("scores.csv", "w", encoding="utf-8-sig", newline="") as f:
10     writer = csv.DictWriter(f, fieldnames=["name", "chinese", "math"])
11     writer.writeheader() # 先写表头
12     writer.writerows(rows)
13
14 # 读
15 with open("scores.csv", encoding="utf-8-sig") as f:
16     for row in csv.DictReader(f):
```

```
17 print(row["name"], row["chinese"])
```

有两个坑要提醒。一是 **newline 要设成空字符串**，否则 Windows 上写出来每行之间会多一个空行。二是编码建议用 **utf-8-sig**，这样用 Excel 打开中文不会乱码。★核心

还有一点，CSV 读出来的所有值都是 **字符串**，哪怕原本是数字。要计算得先转换：

```
1 with open("scores.csv", encoding="utf-8-sig") as f:
2     total = sum(int(row["chinese"]) for row in csv.DictReader(f))
3 print("语文总分", total)
```

这和第一章说的 input 拿到字符串是同一类问题。

两种格式怎么选，看数据形状。有嵌套、每条记录字段可能不一样，用 JSON。一张规整的表，每行字段都相同，用 CSV。

6.6 小结

这一章让数据能留下来。用 open 配 with 读写文件，一定要写 encoding 等于 utf-8，w 模式会清空原文件、追加要用 a。路径用 pathlib 拼接最省心，小文件可以直接用它的 write_text 和 read_text。有结构的数据用 JSON，dumps 和 loads 处理字符串，dump 和 load 处理文件，中文要加 ensure_ascii 等于 False。表格数据用 CSV，注意 newline 设空字符串、编码用 utf-8-sig，而且读出来都是字符串。下一章讲程序出错时怎么办。

本章知识点

- **open** —— 打开文件，r 读、w 写会清空、a 追加
- **with** —— 保证文件用完自动关闭，哪怕中途出错
- **encoding** —— 读写中文一定要写 utf-8，否则容易乱码
- **pathlib** —— 用斜杠拼路径，跨系统通用，还能判断存在与建目录
- **JSON** —— 存有结构的数据，和 Python 的列表字典几乎一样
- **dumps 与 loads** —— 在数据和字符串之间来回转
- **dump 与 load** —— 不带 s 的这一对直接和文件打交道
- **ensure_ascii** —— 设成 False 中文才原样存，否则变成转义码
- **CSV** —— 存规整的表格数据，每行一条记录，Excel 能直接打开
- **newline** —— 写 CSV 时设成空字符串，否则 Windows 上会多空行
- **utf-8-sig** —— 写 CSV 用这个编码，Excel 打开中文不乱码